6.Implementation of Decision tree using sklearn and its parameter tuning

A ***Decision Tree*** is a supervised machine learning algorithm used for classification and regression tasks. It builds a tree-like model of decisions based on features of the data, splitting nodes based on conditions that maximize information gain or Gini index or minimize impurity. Decision Trees are interpretable and useful for understanding the structure of data.

The **Iris dataset** is a well-known dataset in machine learning, containing ***150 samples*** from three species of Iris flowers: *setosa, versicolor, and virginica*. Each sample has four features: *sepal length, sepal width, petal length, and petal width*.

Importing Required Libraries:

- Libraries from **sklearn** are imported for data handling, model training, evaluation, and visualization.
- matplotlib.pyplot is used for plotting the decision tree.
- load_iris: To load the Iris dataset.
- train_test_split: To split the dataset into training and testing subsets.
- GridSearchCV: To perform hyperparameter tuning through grid search.
- DecisionTreeClassifier: To create a Decision Tree model.
- accuracy_score and classification_report: To evaluate model performance.
- tree: For plotting the tree structure.

2. Loading the Iris Dataset

- The Iris dataset is loaded using load_iris().
- X holds the feature data, and y holds the target labels (species of iris).

3. Splitting the Dataset

- Using train_test_split, 90% of the data is used for training and 10% for testing.
- This ensures the model is evaluated on unseen data.

4. Defining a Parameter Grid for Tuning

- **criterion**: Function to measure split quality - 'gini' or 'entropy'.
- **max_depth**: Max depth of the tree (1 to 4).
- **min_samples_split**: Minimum samples to split an internal node.
- **min_samples_leaf**: Minimum samples required at a leaf node.

5. Initializing the Decision Tree Classifier

- An instance of DecisionTreeClassifier is created with a fixed random state

6. Performing Grid Search for Parameter Tuning

- GridSearchCV searches through the parameter grid using cross-validation (cv=3).
- It identifies the best combination of parameters for optimal model performance.

7. Getting the Best Model

- best_estimator_ retrieves the best-performing classifier from the grid search.

8. Fitting the Best Classifier on Training Data

- The best classifier is trained using fit() on X_train and y_train.

9. Making Predictions on the Test Set

- Predictions are made using the best model from the grid search.

11. Printing Classification Report

- classification_report provides precision, recall, and F1-score for each iris species class.

12. Visualizing the Best Decision Tree

- plot_tree from sklearn.tree is used to visualize the tree structure.
- It includes class names, feature names, and colored nodes for interpretability.
- plt.show() displays the tree plot.

Code:

```
# Importing required libraries
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report
import matplotlib.pyplot as plt
from sklearn import tree

# Load the Iris dataset
iris = load_iris()
X = iris.data # Features
y = iris.target # Target labels

# Split the dataset into training and testing sets (90% train, 10% test)
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.1, random_state=42) # 10% test size

# Initialize the Decision Tree classifier with some tuning parameters
clf = DecisionTreeClassifier(max_depth=2, min_samples_split=2, min_samples_leaf=1,
random_state=42)

# Fit the classifier on the training data
clf.fit(X_train, y_train)

# Make predictions on the test set
y_pred = clf.predict(X_test)

# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print(f'Accuracy: {accuracy:.2f}')

# Print classification report
print('Classification Report:')
print(classification_report(y_test, y_pred, target_names=iris.target_names))

# Visualize the Decision Tree
plt.figure(figsize=(10, 8))
tree.plot_tree(clf, filled=True, feature_names=iris.feature_names,class_names=iris.target_names,
rounded=True)
plt.title("Decision Tree Visualization")
plt.show()
```

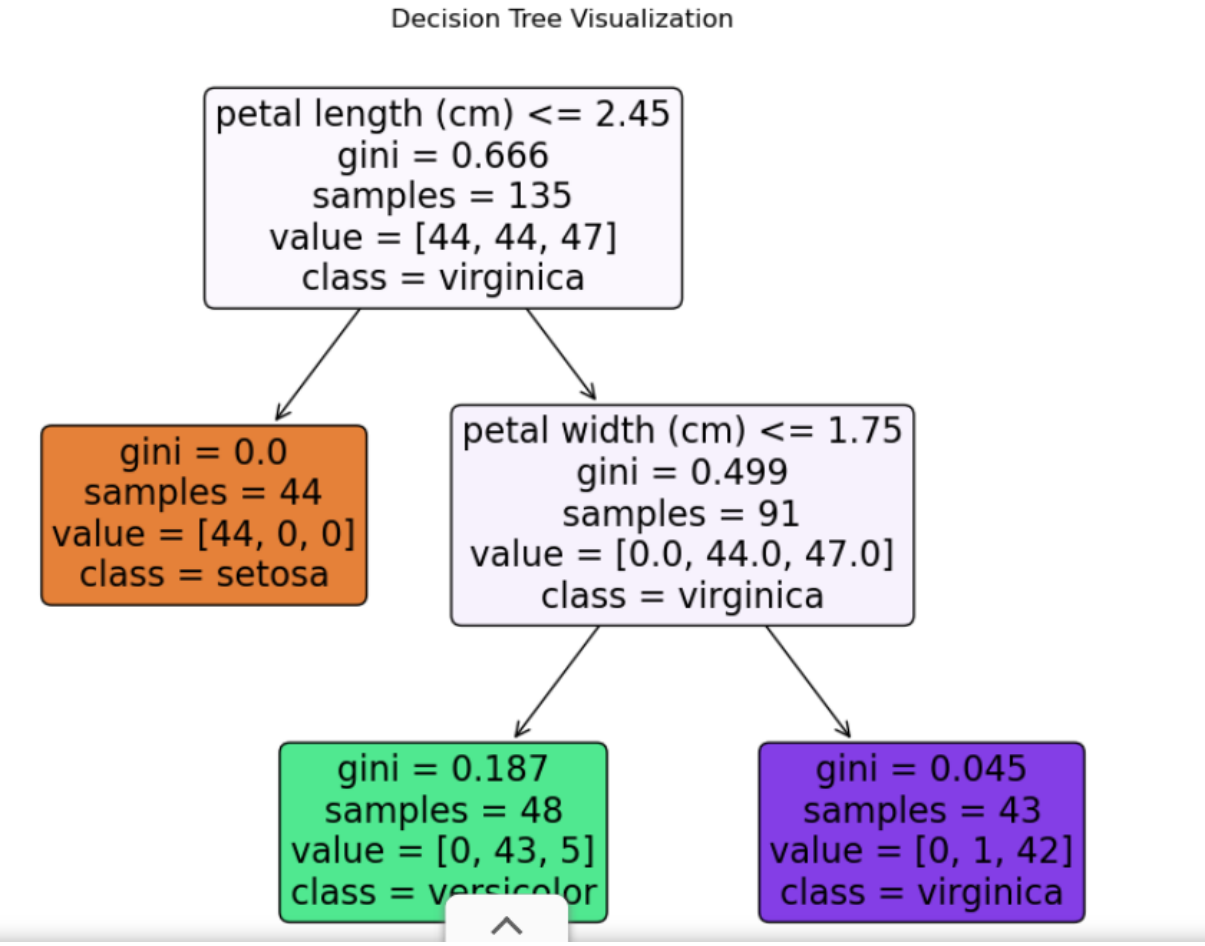
output:

Sample Run:

```
$ python3 Dtree.py
Accuracy: 1.00
```

Classification Report:

	precision	recall	f1-score	support
setosa	1.00	1.00	1.00	6
versicolor	1.00	1.00	1.00	6
virginica	1.00	1.00	1.00	3
accuracy			1.00	15
macro avg	1.00	1.00	1.00	15
weighted avg	1.00	1.00	1.00	15



7.Implementation of KNN using sklearn.

The K-Nearest Neighbors (KNN) algorithm is a supervised machine learning method employed to tackle classification and regression problems.

K-NN algorithm assumes the similarity between the new case/data and available cases and put the new case into the category that is most similar to the available categories.

K-NN algorithm stores all the available data and classifies a new data point based on the similarity. This means when new data appears then it can be easily classified into a well suite category by using K- NN algorithm.

K-NN algorithm can be used for Regression as well as for Classification but mostly it is used for the Classification problems.

In this program, we are going to use ***iris dataset***.And this dataset Split into training(70%) and test set(30%).

Code:

```
from sklearn.neighbors import KNeighborsClassifier

from sklearn.model_selection import train_test_split

from sklearn.datasets import load_iris

import random


data_iris = load_iris()

label_target = data_iris.target_names

print()


print("Sample Data from Iris Dataset")

print("*"*30)

for i in range(10):

        rn = random.randint(0,120)

        print(data_iris.data[rn],"===>",label_target[data_iris.target[rn]])


X = data_iris.data

y = data_iris.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.3,random_state=1)


print("The Training dataset length: ",len(X_train))

print("The Testing dataset length: ",len(X_test))
```

```
try:
        nn = int(input("Enter number of neighbors :"))
        knn = KNeighborsClassifier(nn)
        knn.fit(X_train, y_train)
        # to display the score
        print("The Score is :",knn.score(X_test, y_test))
        # To get test data from the user
        test_data = input("Enter Test Data :").split(",")
        for i in range(len(test_data)):
                test_data[i] = float(test_data[i])
        print()
        v = knn.predict([test_data])
        print("Predicted output is :",label_target[v])
except:
        print("Please supply valid input.....")
```

output:

```
Sample Run:
--------------
$ python3 Knn.py
Sample Data from Iris Dataset
******************************
[6.1 2.8 4.  1.3] ===> versicolor
[4.6 3.1 1.5 0.2] ===> setosa
[5.8 2.6 4.  1.2] ===> versicolor
[6.9 3.2 5.7 2.3] ===> virginica
[6.7 3.1 4.7 1.5] ===> versicolor
[4.7 3.2 1.3 0.2] ===> setosa
[5.  3.  1.6 0.2] ===> setosa
[6.4 2.9 4.3 1.3] ===> versicolor
[5.5 2.4 3.7 1. ] ===> versicolor
```

[5.8 2.6 4. 1.2] ===> versicolor

The Training dataset length: 105

The Testing dataset length: 45

Enter number of neighbors :9

The Score is : 0.9777777777777777

Enter Test Data :5.3,2.2,3.2,1.5

Predicted output is : ['versicolor']

8. **Implementation of Logistic Regression using sklearn.**

Logistic regression is one of the most popular Machine Learning algorithms, which comes under the Supervised Learning technique. It is used for predicting the categorical dependent variable using a given set of independent variables. Logistic regression predicts the output of a categorical dependent variable. Therefore the outcome must be a categorical or discrete value. It can be either Yes or No, 0 or 1, true or False, etc. but instead of giving the exact value as 0 and 1, it gives the probabilistic values which lie between 0 and 1.

In this implementation, we use "Study Hours" as the independent variable and "Exam Results (0 or 1)" as the dependent variable.

Code:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

from sklearn.linear_model import LogisticRegression

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report


# Step 1: Load the dataset

data = pd.read_csv("study_hours.csv")  # Ensure this file exists in the same directory

X = data[['Study Hours']].values

y = data['Exam Result'].values


# Step 2: Train-test split
```

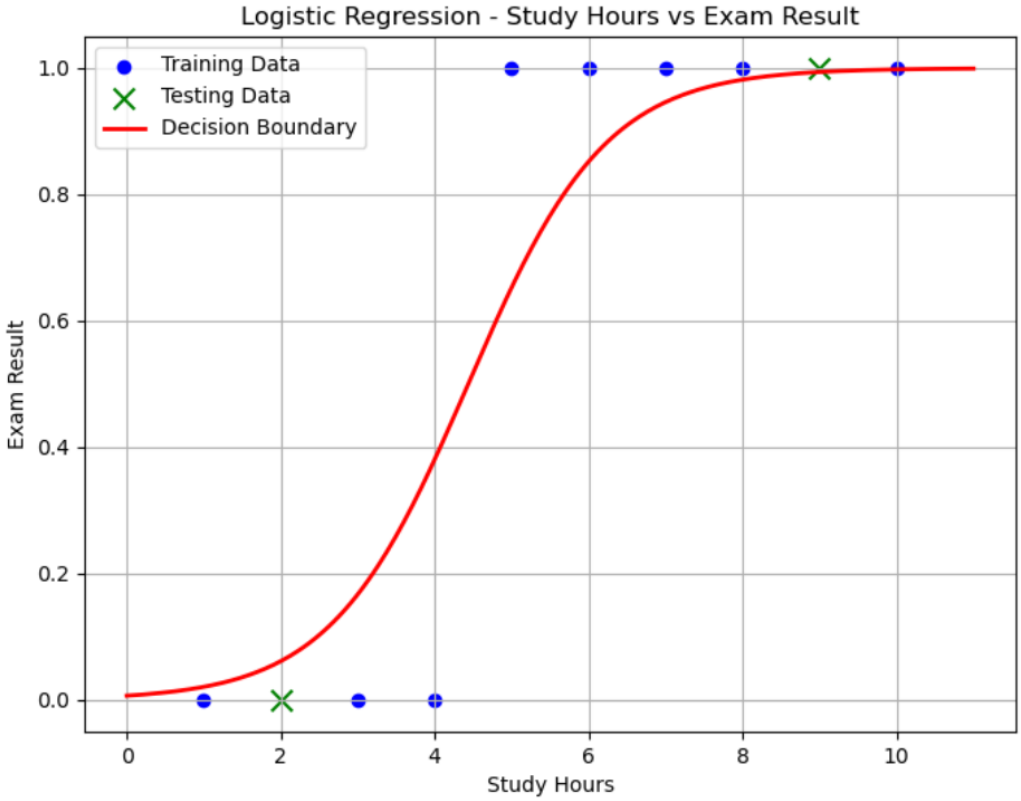
```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Step 3: Train logistic regression model
model = LogisticRegression()
model.fit(X_train, y_train)

# Step 4: Predict and evaluate
y_pred = model.predict(X_test)

# Step 5: Print results
print(f"Accuracy: {accuracy_score(y_test, y_pred):.1f}")
print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

# Step 6: Plotting decision boundary
X_range = np.linspace(X.min() - 1, X.max() + 1, 300).reshape(-1, 1)
y_prob = model.predict_proba(X_range)[:, 1]

plt.figure(figsize=(8, 6))
plt.scatter(X_train, y_train, color='blue', label='Training Data')
plt.scatter(X_test, y_test, color='green', marker='x', s=100, label='Testing Data')
plt.plot(X_range, y_prob, color='red', linewidth=2, label='Decision Boundary')
plt.xlabel("Study Hours")
plt.ylabel("Exam Result")
plt.title("Logistic Regression - Study Hours vs Exam Result")
plt.legend()
plt.grid(True)
```

```
plt.show()
```

output:

Sample Run:

```
$ python3 Logistic_Regression.py
Accuracy: 1.0

Confusion Matrix:
[[1 0]
 [0 1]]

Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00         1
           1       1.00      1.00      1.00         1

    accuracy                           1.00         2
   macro avg       1.00      1.00      1.00         2
weighted avg       1.00      1.00      1.00         2
```



9. **Implementation of K-Means Clustering.**

K-Means is a popular unsupervised machine learning algorithm used for partitioning a dataset into K clusters based on the similarity of their features. It works by minimizing the variance within each cluster and is widely used in applications such as market segmentation, image compression, and pattern recognition.

K means clustering, assigns data points to one of the K clusters depending on their distance from the center of the clusters. It starts by randomly assigning the clusters centroid in the space. Then each data point assign to one of the cluster based on its distance from centroid of the cluster. After assigning each point to one of the cluster, new cluster centroids are assigned. This process runs iteratively until it finds good cluster. In the analysis we assume that number of cluster is given in advanced and we have to put points in one of the group.

ALGORITHM:

1. **Initialization:**
    - Choose the number of clusters (k) to divide the data into.
    - Initialize the centroids (cluster centers) randomly or using specific techniques.
2. **Assignment:**
    - Assign each data point to the nearest centroid based on the distance (e.g., Euclidean distance).
    - Each data point is labeled as belonging to one of the k clusters.
3. **Update:**

- Recalculate the centroids of the clusters by averaging the coordinates of all data points assigned to each cluster.

4. **Repeat:**
    - Repeat the Assignment and Update steps iteratively until the centroids stabilize (i.e., no significant changes in their positions) or a predefined number of iterations is reached.

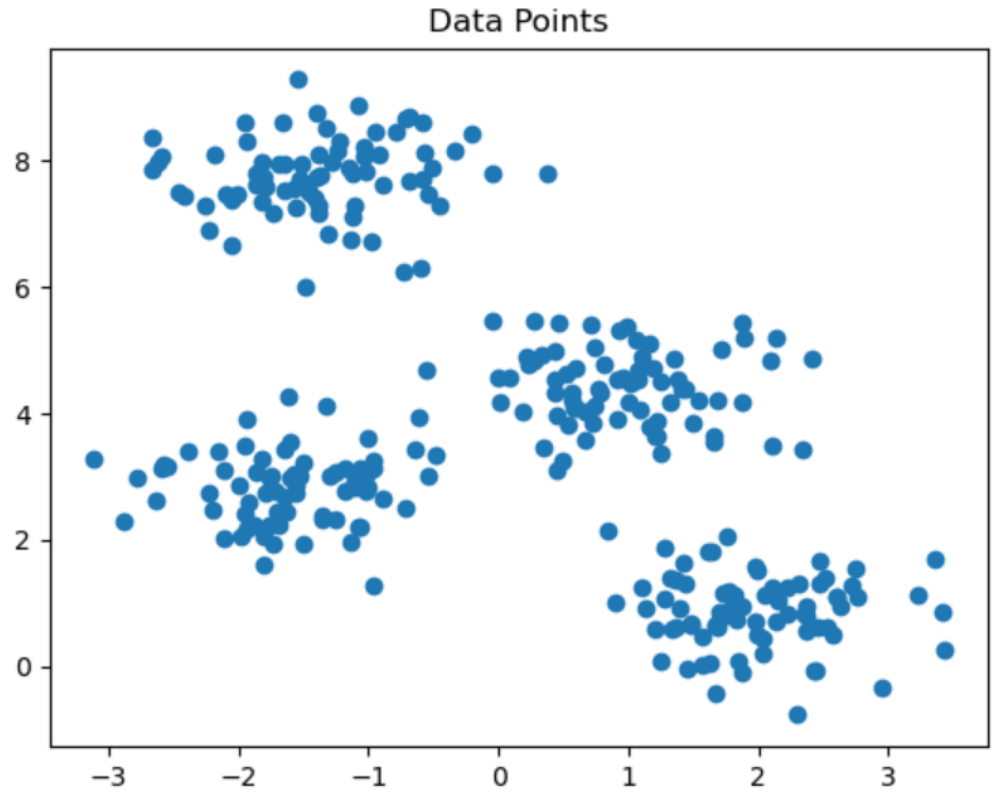
Steps for Program:

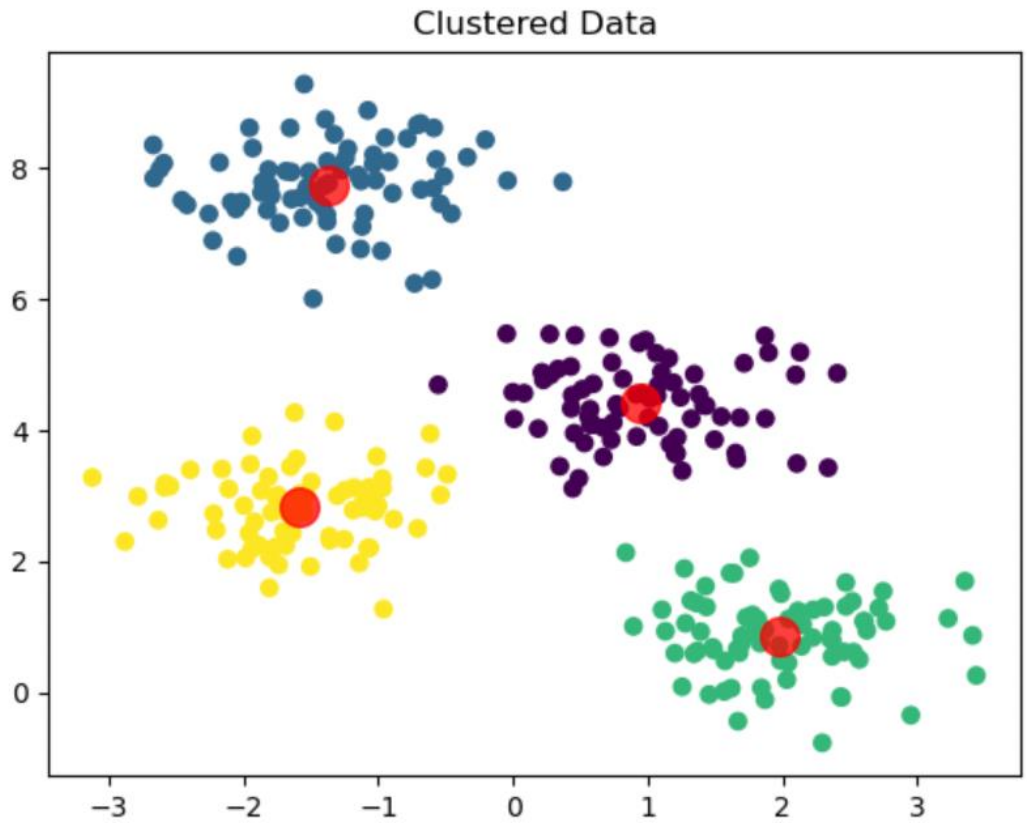
1. **Import Necessary Libraries:**
    - numpy: For numerical computations.
    - matplotlib.pyplot: For visualizing the data points and clustering results.
    - KMeans from sklearn.cluster: To implement the K-Means algorithm.
    - make_blobs from sklearn.datasets: To generate synthetic data for clustering.
2. **Generate Synthetic Data:**
    - The make_blobs function generates 300 data points with 4 clusters. The points are distributed with a standard deviation of 0.60 around the cluster centers.
    - The dataset consists of:
      Features (X): A 2D array representing the data points' coordinates.
3. **Visualize Raw Data:**
    - A scatter plot is created to visualize the data points before clustering.
4. **Apply K-Means Algorithm:**
    - The K-Means model is initialized with n_clusters=4 to specify 4 clusters.
    - The model is fit to the data using the fit method.
5. **Retrieve Cluster Information:**
    - **Cluster Centers:** The coordinates of the centroids for each cluster are stored in kmeans.cluster_centers_.
    - **Cluster Labels:** Each data point is assigned a cluster label using kmeans.labels_.
6. **Visualize Clustered Data:**
    - A scatter plot shows the data points grouped into clusters, with colors representing different clusters.
    - Cluster centroids are displayed as red points to highlight their locations.

Code:

```
#Necessary Libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs

#Generate Data
X, _ = make_blobs(n_samples = 300, centers = 4 , cluster_std = 0.60,random_state = 0)

#Plot the data points
plt.scatter(X[:,0],X[:,1])
plt.title("Data Points")
plt.show()

#KMeans clustering
kmeans = KMeans(n_clusters = 4)
kmeans.fit(X)

#Getting the cluster centers and labels
centers = kmeans.cluster_centers_ #returns the coordinates of the

#cluster center found by the algorithm
labels = kmeans.labels_ #Returns the labels for each data point , indicating which cluster it has

#been assigned to.
#Plot the clustered data
plt.scatter(X[:,0],X[:,1],c = labels)
plt.scatter(centers[:,0],centers[:,1],alpha = 0.75,s=200,color = 'red')
plt.title("Clustered Data")
plt.show()
```

output:





10. **Mini Project : Performance Analysis of Classification Algorithms on the Iris Dataset.**

Performance Analysis of Classification Algorithms on the Iris Dataset

Objective:

To evaluate and compare the performance of various classification algorithms such as:

- Logistic Regression
- Decision Tree
- K-Nearest Neighbors (KNN)
- Support Vector Machine (SVM)
- Random Forest

using performance metrics like accuracy, confusion matrix, precision, recall, and F1-score.

Code:

```
import numpy as np

import pandas as pd

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import StandardScaler

from sklearn.linear_model import LogisticRegression

from sklearn.tree import DecisionTreeClassifier

from sklearn.neighbors import KNeighborsClassifier

from sklearn.svm import SVC

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import classification_report, confusion_matrix, accuracy_score


# Load dataset

iris = load_iris()

X = iris.data

y = iris.target


# Train-test split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

```
# Feature scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Define models
models = {
    "Logistic Regression": LogisticRegression(),
    "Decision Tree": DecisionTreeClassifier(),
    "K-Nearest Neighbors": KNeighborsClassifier(),
    "Support Vector Machine": SVC(),
    "Random Forest": RandomForestClassifier()
}

# Train, Predict, and Evaluate
for name, model in models.items():
    print(f"\nModel: {name}")
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    print("Accuracy:", accuracy_score(y_test, y_pred))
    print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
    print("Classification Report:\n", classification_report(y_test, y_pred))
```

output:

```
Sample Run:
--------------
$ python3 mini_project.py
Model: Logistic Regression
Accuracy: 1.0
Confusion Matrix:
 [[19  0  0]
```

```
 [ 0 13  0]
 [ 0  0 13]]
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        19
           1       1.00      1.00      1.00        13
           2       1.00      1.00      1.00        13

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45
```

```
Model: Decision Tree
Accuracy: 1.0
Confusion Matrix:
 [[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]
Classification Report:
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        19
           1       1.00      1.00      1.00        13
           2       1.00      1.00      1.00        13

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45
```

Model: K-Nearest Neighbors

Accuracy: 1.0

Confusion Matrix:

```
[[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]
```

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	19
1	1.00	1.00	1.00	13
2	1.00	1.00	1.00	13
accuracy			1.00	45
macro avg	1.00	1.00	1.00	45
weighted avg	1.00	1.00	1.00	45

Model: Support Vector Machine

Accuracy: 1.0

Confusion Matrix:

```
[[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]
```

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	19
1	1.00	1.00	1.00	13
2	1.00	1.00	1.00	13

accuracy			1.00	45
macro avg	1.00	1.00	1.00	45
weighted avg	1.00	1.00	1.00	45

Model: Random Forest

Accuracy: 1.0

Confusion Matrix:

```
[[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]
```

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	19
1	1.00	1.00	1.00	13
2	1.00	1.00	1.00	13
accuracy			1.00	45
macro avg	1.00	1.00	1.00	45
weighted avg	1.00	1.00	1.00	45